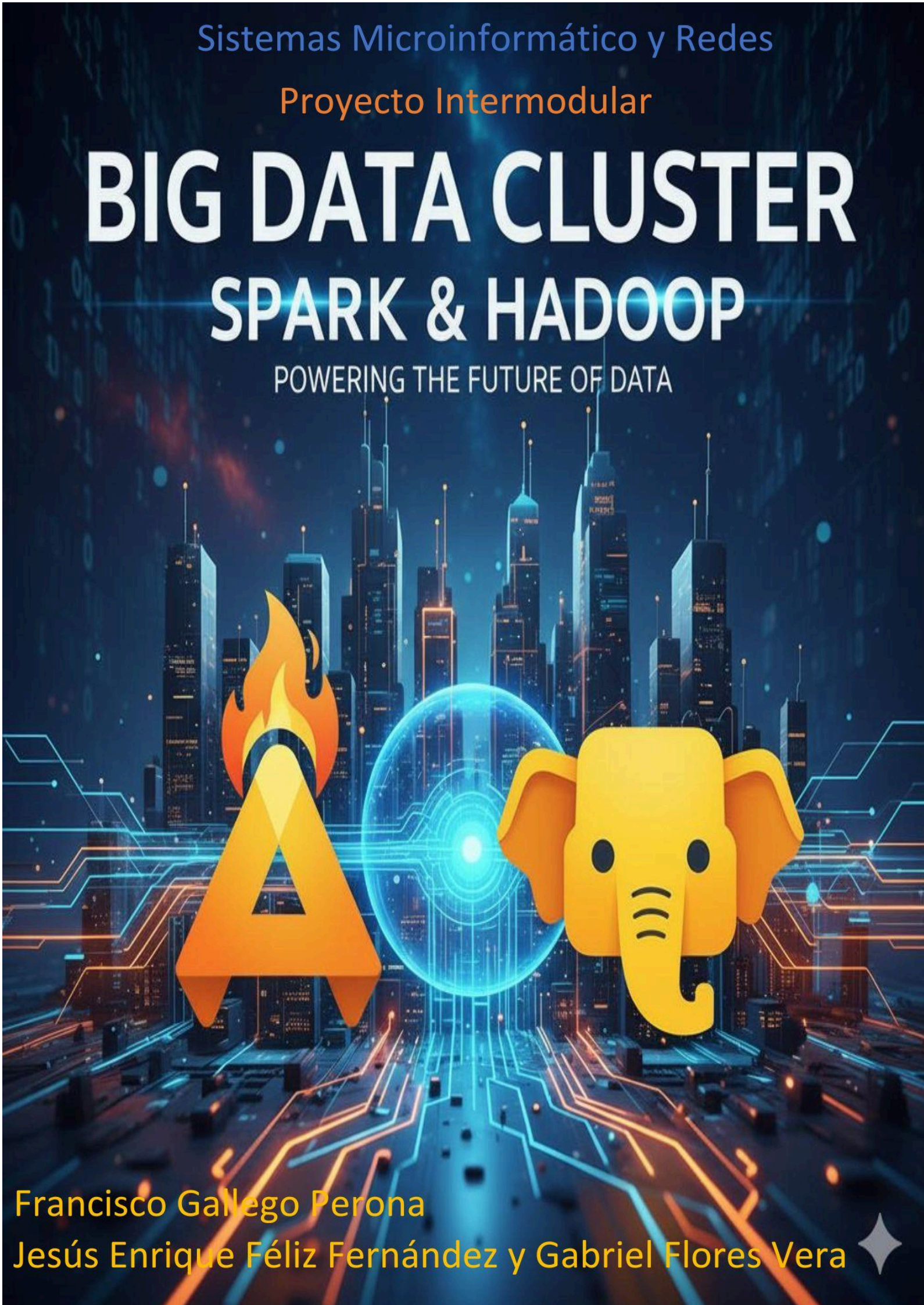
Sistemas Microinformático y Redes

Proyecto Intermodular

BIG DATA CLUSTER

SPARK & HADOOP

POWERING THE FUTURE OF DATA



Francisco Gallego Perona

Jesús Enrique Féliz Fernández y Gabriel Flores Vera

ÍNDICE

1-Resumen

Este proyecto tiene como objetivo la creación de un cluster para el procesamiento de grandes volúmenes de datos (Big Data), Utilizando como motor para analizar y procese los datos Apache Spark todo esto hecho en un contenedor (Docker)

Lo hemos hecho en Docker para crear un contenedor para facilitar la instalación de spark sin la necesidad de configurar varias máquinas físicas

Utilizamos Apache Spark para procesar todos los datos de forma rápida y eficiente. Se ejecutaron varias tareas de análisis sobre los datos

En conclusión, con este proyecto se permite comprender los conceptos básicos de Big Data, el funcionamiento de Spark y las ventajas del uso de Docker para montar el cluster.

2-Introducción

2.1 Contexto y justificación del proyecto.

Actualmente, la cantidad de datos generados ha hecho necesario el uso de tecnologías capaces de procesar grandes volúmenes de datos. El Big Data surge como respuesta a esta necesidad, permitiendo analizar datos que no podrían ser tratados con herramientas tradicionales.

El uso de Apache Spark permite comprender cómo se procesan datos de forma distribuida, mientras que Docker facilita la creación de entornos de trabajo estables y reproducibles, simplificando la instalación y configuración de los servicios necesarios.

2.2 Objetivos del proyecto.

El objetivo es hacer un cluster con Apache Spark desplegado con Docker para analizar grandes volúmenes de datos (Big Data)

Los objetivos específicos son:

- Configurar y ejecutar Apache Spark en contenedores Docker.
- Realizar tareas sencillas de procesamiento y análisis de datos.
- Comprender las ventajas del uso de Docker para el despliegue de aplicaciones.

2.3 Alcance y limitaciones del trabajo.

El alcance del proyecto se centra en la creación de un pequeño cluster de Big Data mediante contenedores Docker y con el motor de procesamiento de apache spark .

Las limitaciones del proyecto son:

- No se utiliza un cluster real con múltiples máquinas físicas.
- El volumen de datos procesado es limitado.
- no hay bases de datos grandes sin imágenes.
- no puede funcionar en equipos antiguos
- hay que pasar tiempo buscando constructores o docker-compose que estén actualizados o las imágenes aun sigan funcionando.

3-Análisis y contextualización de empresa/s del sector

3.1-Caracterización de la empresa del sector

Big Cine es una empresa perteneciente al sector tecnológico–audiovisual, especializada en la gestión, almacenamiento y análisis de grandes volúmenes de datos (big data) relacionados con el cine y el contenido audiovisual.

El sector audiovisual vinculado al big data está en crecimiento debido al aumento del consumo digital y la necesidad de analizar datos para mejorar la toma de decisiones. Ofrece oportunidades de innovación y expansión, aunque presenta riesgos como la alta competencia y la dependencia tecnológica.

3.2-Productos y servicios

Big Cine ofrece servicios de almacenamiento, análisis y explotación de datos cinematográficos, elaboración de informes, sistemas de recomendación y consultoría para empresas del sector audiovisual.

Su público objetivo son productoras, plataformas de streaming, instituciones culturales y educativas.
Se diferencia de la competencia por su especialización exclusiva en datos cinematográficos y su análisis avanzado.

3.3-Relación con los Objetivos de Desarrollo Sostenible (ODS)

Big Cine se relaciona con:
ODS 9 (Innovación e infraestructura) mediante el uso de tecnología avanzada.
ODS 8 (Trabajo decente) al generar empleo cualificado.
ODS 12 (Producción responsable) al optimizar recursos mediante análisis de datos.
ODS 4 (Educación de calidad) apoyando la investigación y el aprendizaje.

3.4-Identificación de los riesgos laborales

Los principales riesgos laborales son ergonómicos, psicosociales, fatiga visual y sedentarismo, derivados del trabajo prolongado con equipos informáticos. Es necesario aplicar medidas preventivas como ergonomía adecuada y pausas activas.

3.5-Conclusiones

Big Cine representa un modelo de empresa innovadora que combina cine y big data en un sector en expansión. Como oportunidades de mejora destacan el refuerzo de la seguridad de datos, la mejora del bienestar laboral y el desarrollo de servicios educativos y sostenibles.

4-Desarrollo del proyecto

4.1 Qué es HDFS

HDFS es el sistema de gestión de almacenamiento principal de Hadoop algo que se puede sacar del significado de las siglas Hadoop Distributed File System (Sistema de distribución de ficheros de hadoop). HDFS es bastante eficiente en lo que trata a Big Data ya que divide los datos en bloques y los replica en todos los datanode gracias a eso es también es a prueba de fallos y tiene una recuperación fácil de hardware al tener copias de los datos en todos los datanodes

Estos son los dos tipos de nodos en HDFS

Namenode: Se encarga de saber dónde están los archivos, los permisos que tienen y la ubicación del bloque. También controla el acceso de los clientes a los archivos y realiza operaciones como abrir, cerrar y renombrar directorios y archivos

Datanode: Son los que se encargan de la creación, eliminación y replicación de bloques, además de las solicitudes de escritura y lectura.

4.2 Qué es SPARK

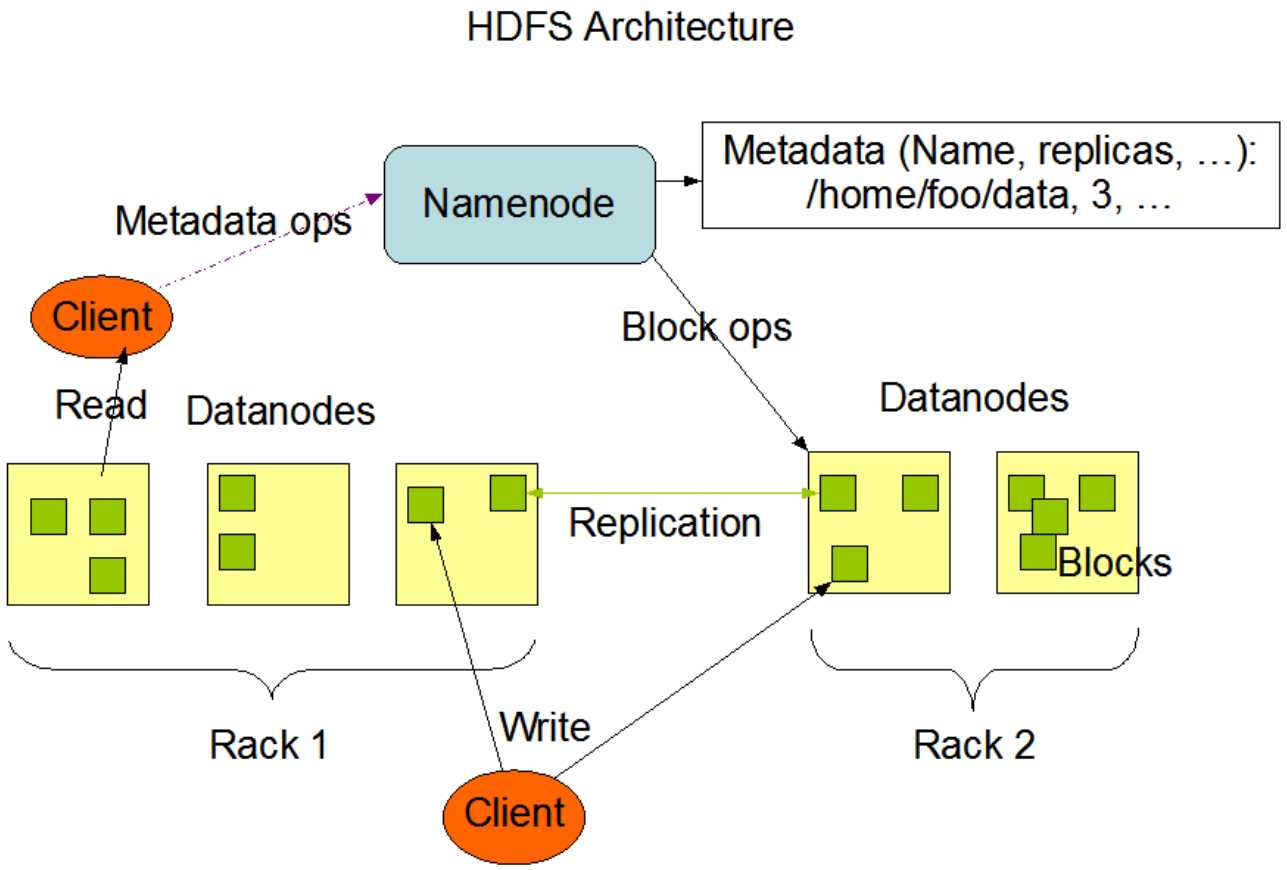
SPARK es el motor de procesamiento de datos el puede gestionar grandes cantidades de datos por eso se utiliza bastante en Big Data y se adapta bien a PySpark. En general SPARK gracias a su velocidad computacional, escalabilidad y la programabilidad necesaria para Big Data, específicamente para la transmisión de datos, datos gráficos, análisis, machine learning, procesamiento de datos a gran escala y aplicaciones de inteligencia artificial (IA).

Estos son los tipos de nodos que tiene SPARK

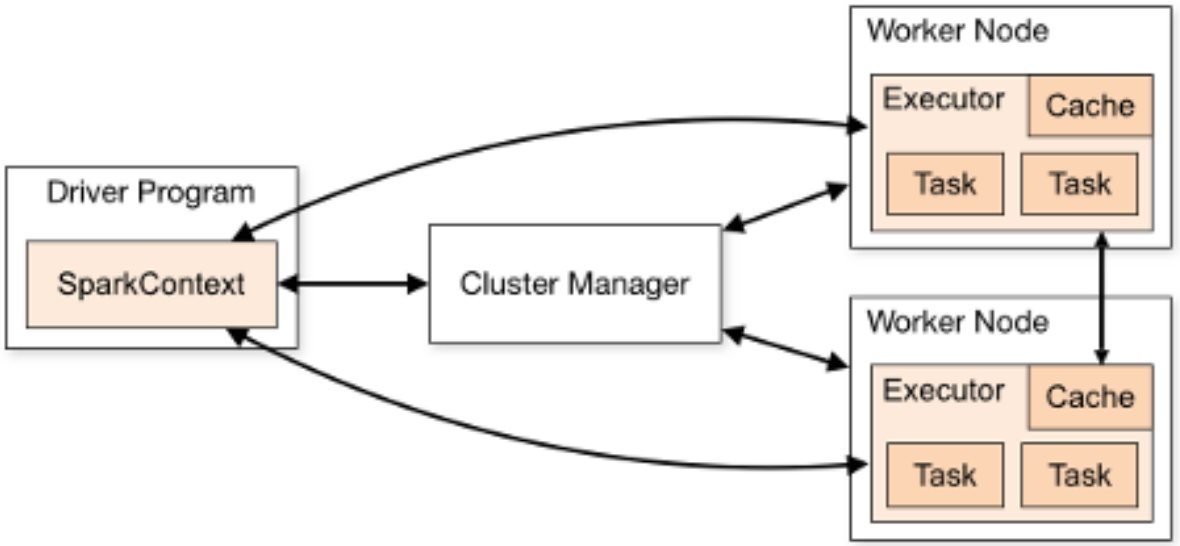
Spark-master: Es el nodo central en spark el cual se encarga de administrar los recursos y cordinar a los workers. El es el que registra a los workers, asigna los recursos y monitorea aplicaciones

Spark-worker: Se encarga de que recibe la tarea del master y la ejecuta, procesa los datos y reporta su estado y uso de recursos al Master.

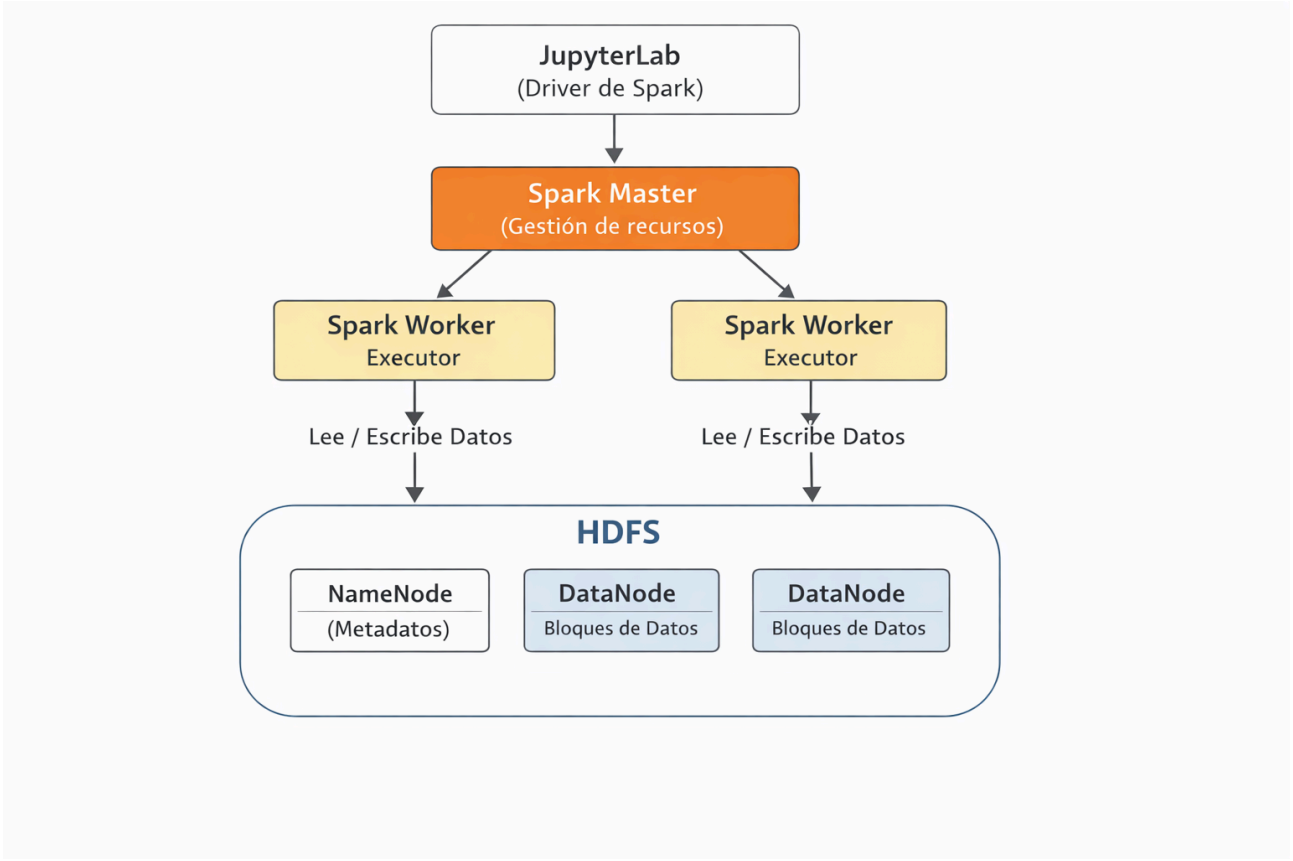
HDFS


HDFS Architecture
Namenode
Metadata (Name, replicas, …):
/home/foo/data, 3, …
Metadata ops
Client
Block ops
Read
Datanodes
Datanodes
Replication
Blocks
Rack 1
Write
Rack 2
Client

SPARK


Worker Node
Executor
Cache
Task
Task
Driver Program
SparkContext
Cluster Manager
Worker Node
Executor
Cache
Task
Task

JupyterLab + Spark + HDFS



4.3 creación del contenedor de HDFS

Aquí tenemos el .yml para montar el contenedor de HDFS aquí ponemos el namenode y el datanode y lo ponemos en una red docker llamada bigdata_movies

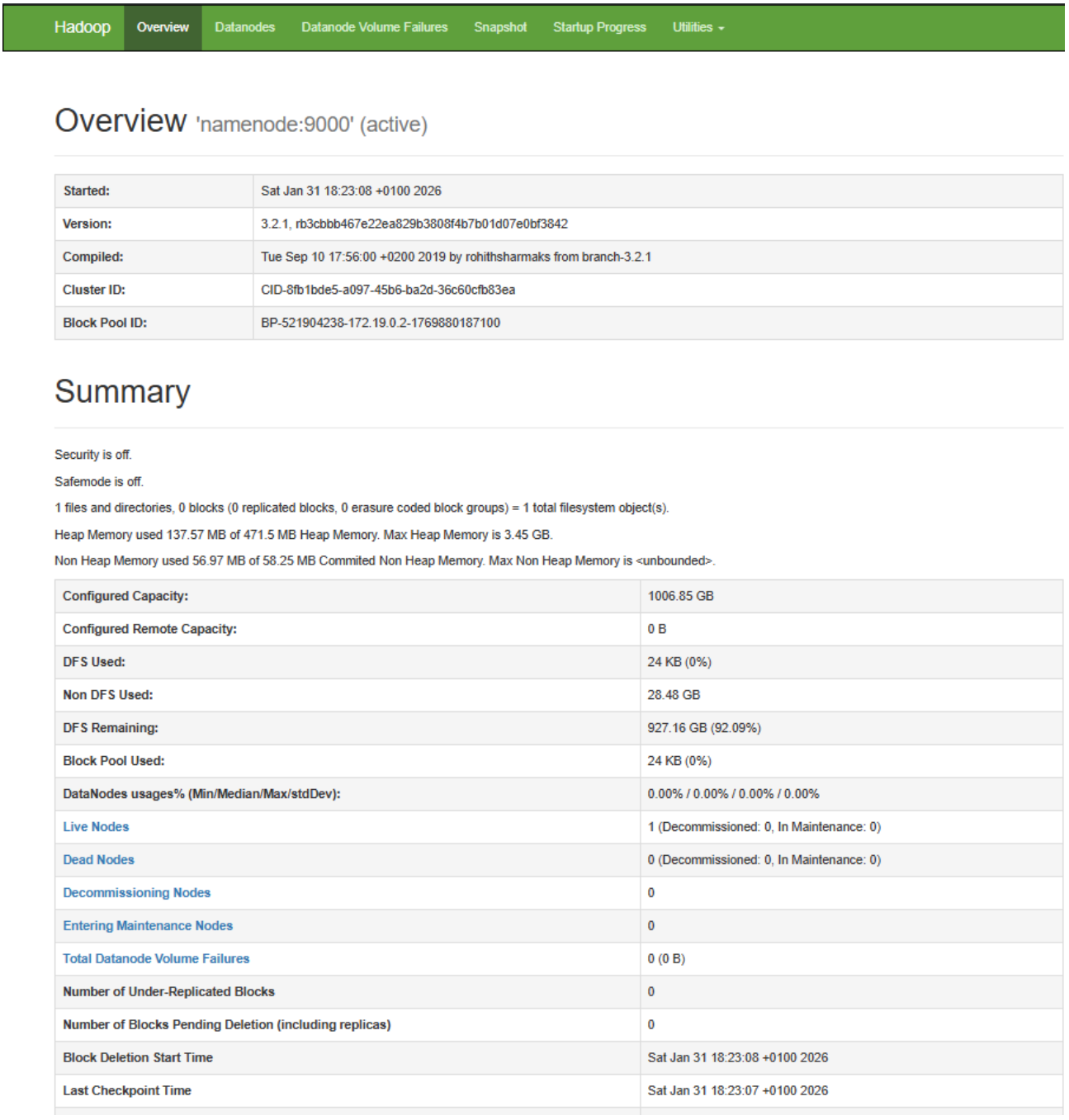
```
  GNU nano 7.2
services:
  namenode:
    image: bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8
    container_name: namenode
    ports:
      - "9870:9870"
      - "9000:9000"
    environment:
      - CLUSTER_NAME=hdfs-cluster
      - CORE_CONF_fs_defaultFS=hdfs://namenode:9000
      - CORE_CONF_hadoop_staticuser_user=root
      - HDFS_CONF_dfs_replication=1
    volumes:
      - namenode:/hadoop/dfs/name
    networks:
      - bigdata_movies

  datanode:
    image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    container_name: datanode
    environment:
      - CLUSTER_NAME=hdfs-cluster
      - CORE_CONF_fs_defaultFS=hdfs://namenode:9000
      - CORE_CONF_hadoop_http_staticuser_user=root
      - HDFS_CONF_dfs_replication=1
    depends_on:
      - namenode
    volumes:
      - datanode:/hadoop/dfs/data
    networks:
      - bigdata_movies

volumes:
  namenode:
  datanode:

networks:
  bigdata_movies:
    external: true
```

Docker compose up para que construya el contenedor .

```
alusmr@adrianl:~/Documentos/HDFS$ docker compose up -d
WARN[0000] /home/alusmr/Documentos/HDFS/docker-compose.yml: the attribute `version` is obsolete, it will
 be ignored, please remove it to avoid potential confusion
[+] Running 18/18
 ✔ namenode Pulled                                                                                  56.6s
   ✔ facffb3a6de3 Pull complete                                                                     54.8s
   ✔ c71a6df73788 Pull complete                                                                     54.9s
   ✔ 73b8c0ccb707 Pull complete                                                                     54.9s
 ✔ datanode Pulled                                                                                  56.6s
   ✔ 3192219afd04 Pull complete                                                                     10.3s
   ✔ 7127a1d8cced Pull complete                                                                     28.6s
   ✔ 883a89599900 Pull complete                                                                     28.7s
   ✔ 77920a3e82af Pull complete                                                                     28.7s
   ✔ 92329e81aec4 Pull complete                                                                     54.6s
   ✔ f373218fec59 Pull complete                                                                     54.6s
   ✔ aa53513fe997 Pull complete                                                                     54.7s
   ✔ 8b1800105b98 Pull complete                                                                     54.7s
   ✔ c3a84a3e49c8 Pull complete                                                                     54.7s
   ✔ a65640a64a76 Pull complete                                                                     54.8s
   ✔ 3ca2ec07878c Pull complete                                                                     54.8s
   ✔ 26c2dd45430e Pull complete                                                                     54.9s
   ✔ 13c9c87a46cb Pull complete                                                                     54.9s
[+] Running 5/5
 ✔ Network hdfs_default    Created                                                                   0.1s
 ✔ Volume "hdfs_namenode"  Created                                                                   0.0s
 ✔ Volume "hdfs_datanode"  Created                                                                   0.0s
 ✔ Container namenode      Started                                                                   0.4s
 ✔ Container datanode      Started                                                                   0.3s
alusmr@adrianl:~/Documentos/HDFS$ docker exec -it namenode bash
root@fc23dcc9e3b4:/# hdfs dfs -ls /
root@fc23dcc9e3b4:/# hdfs dfs -mkdir /cinebeni
root@fc23dcc9e3b4:/# hdfs dfs -ls /
Found 1 items
drwxr-xr-x   - root supergroup          0 2026-01-22 18:18 /cinebeni
root@fc23dcc9e3b4:/# exit
exit
alusmr@adrianl:~/Documentos/HDFS$ docker network create bigcine-net
9c0bc28e952277f707963aaee877a150319404da29e6f71be2d971d5cdb1c7c3
alusmr@adrianl:~/Documentos/HDFS$
```

Vemos que tenemos el contenedor creado con los puerto accedemos a la interfaz grafica

Esta es la interfaz que vemos si vamos al puerto que nos aparece en docker y vemos que nos sale un nodo vivo eso significa que el datanode que hemos hecho se ha conectado correctamente. esto no salio a la segunda porque a la primera no se podia conectar



subir archivos a HDFS mediante comandos

```
alusmr@adrianl:~/Documentos/HDFS$ ls
actors.csv  docker-compose.yml  languages.csv  releases.csv  themes.csv
crew.csv    genres.csv          posters.csv    studios.csv
alusmr@adrianl:~/Documentos/HDFS$ docker cp themes.csv namenode:/tmp/themes.csv
Successfully copied 5.56MB to namenode:/tmp/themes.csv
alusmr@adrianl:~/Documentos/HDFS$ docker cp studios.csv namenode:/tmp/studios.csv
Successfully copied 18.3MB to namenode:/tmp/studios.csv
alusmr@adrianl:~/Documentos/HDFS$ docker cp releases.csv namenode:/tmp/releases.csv
Successfully copied 51.2MB to namenode:/tmp/releases.csv
alusmr@adrianl:~/Documentos/HDFS$ docker cp languages.csv namenode:/tmp/languages.csv
Successfully copied 28.6MB to namenode:/tmp/languages.csv
alusmr@adrianl:~/Documentos/HDFS$ docker cp posters.csv namenode:/tmp/posters.csv
Successfully copied 94.9MB to namenode:/tmp/posters.csv
alusmr@adrianl:~/Documentos/HDFS$ docker cp genres.csv namenode:/tmp/genres.csv
Successfully copied 17.9MB to namenode:/tmp/genres.csv
alusmr@adrianl:~/Documentos/HDFS$ docker cp crew.csv namenode:/tmp/crew.csv
Successfully copied 162MB to namenode:/tmp/crew.csv
alusmr@adrianl:~/Documentos/HDFS$ docker cp movies.csv namenode:/tmp/movies.csv
Successfully copied 254MB to namenode:/tmp/movies.csv
alusmr@adrianl:~/Documentos/HDFS$ docker exec -it namenode hdfs dfs -put /tmp/movies.csv /data/
2026-02-02 20:13:23,311 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted
= false, remoteHostTrusted = false
2026-02-02 20:13:23,636 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted
= false, remoteHostTrusted = false
alusmr@adrianl:~/Documentos/HDFS$ docker exec -it namenode hdfs dfs -put /tmp/genres.csv /data/
2026-02-02 20:13:34,064 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted
= false, remoteHostTrusted = false
alusmr@adrianl:~/Documentos/HDFS$ docker exec -it namenode hdfs dfs -put /tmp/languages.csv /data/
2026-02-02 20:13:44,830 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted
= false, remoteHostTrusted = false
^[[Aalusmr@adrianl:~/Documentos/HDFS$ docker exec -it namenode hdfs dfs -put /tmp/themes.csv /data/
2026-02-02 20:13:55,402 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted
= false, remoteHostTrusted = false
alusmr@adrianl:~/Documentos/HDFS$ docker exec -it namenode hdfs dfs -put /tmp/posters.csv /data/
2026-02-02 20:14:07,965 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted
= false, remoteHostTrusted = false
alusmr@adrianl:~/Documentos/HDFS$ docker exec -it namenode hdfs dfs -put /tmp/studio.csv /data/
put: `/tmp/studio.csv': No such file or directory
alusmr@adrianl:~/Documentos/HDFS$ docker exec -it namenode hdfs dfs -put /tmp/studios.csv /data/
2026-02-02 20:14:20,410 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted
= false, remoteHostTrusted = false
alusmr@adrianl:~/Documentos/HDFS$ docker exec -it namenode hdfs dfs -put /tmp/releases.csv /data/
2026-02-02 20:14:38,718 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted
= false, remoteHostTrusted = false
alusmr@adrianl:~/Documentos/HDFS$ docker exec -it namenode hdfs dfs -put /tmp/crew.csv /data/
2026-02-02 20:14:51,121 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted
= false, remoteHostTrusted = false
2026-02-02 20:14:51,480 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted
= false, remoteHostTrusted = false
alusmr@adrianl:~/Documentos/HDFS$
```

Creación de APACHE Spark

Si buscamos un docker compose para construir el contenedor hay que verificar que las imágenes del compose sigan existiendo. Con este tuvimos más problemas que con HDFS ya que siempre daba problemas al construirlo porque no encontraba las imágenes o no eran compatibles y luego que el master no conocía a los workers

```
  GNU nano 8.4                                  docker-compose.yml
volumes:
  shared-workspace:
    name: "cluster_tfg"
services:
  spark-master:
    image: andreper/spark-master
    container_name: spark-master
    hostname: spark-master
    environment:
      - SPARK_MODE=master
    ports:
      - "7077:7077"
      - "8080:8080"
    volumes:
      - shared-workspace:/opt/workspace
    networks:
      - bigdata_movies

  spark-worker-1:
    image: andreper/spark-worker
    container_name: spark-worker-1
    depends_on:
      - spark-master
    environment:
      - SPARK_MODE=worker
      - SPARK_MASTER_URL=spark://spark-master:7077
      - SPARK_WORKER_CORES=2
      - SPARK_WORKER_MEMORY=1024m
    ports:
      - "8081:8081"
    volumes:
      - shared-workspace:/opt/workspace
    networks:
      - bigdata_movies


  spark-worker-2:
    image: andreper/spark-worker
    container_name: spark-worker-2
    depends_on:
      - spark-master
    environment:
      - SPARK_MODE=worker
      - SPARK_MASTER_URL=spark://spark-master:7077
      - SPARK_WORKER_CORES=2
      - SPARK_WORKER_MEMORY=1024m
    ports:
      - "8082:8081"
    volumes:
      - shared-workspace:/opt/workspace
    networks:
      - bigdata_movies


  jupyterlab:
    image: andreper/jupyterlab
    container_name: jupyterlab
    depends_on:
      - spark-master
    environment:
      - SPARK_MASTER=spark://spark-master:7077
    ports:
      - "8888:8888"
    volumes:
      - shared-workspace:/opt/workspace
    networks:
      - bigdata_movies
```

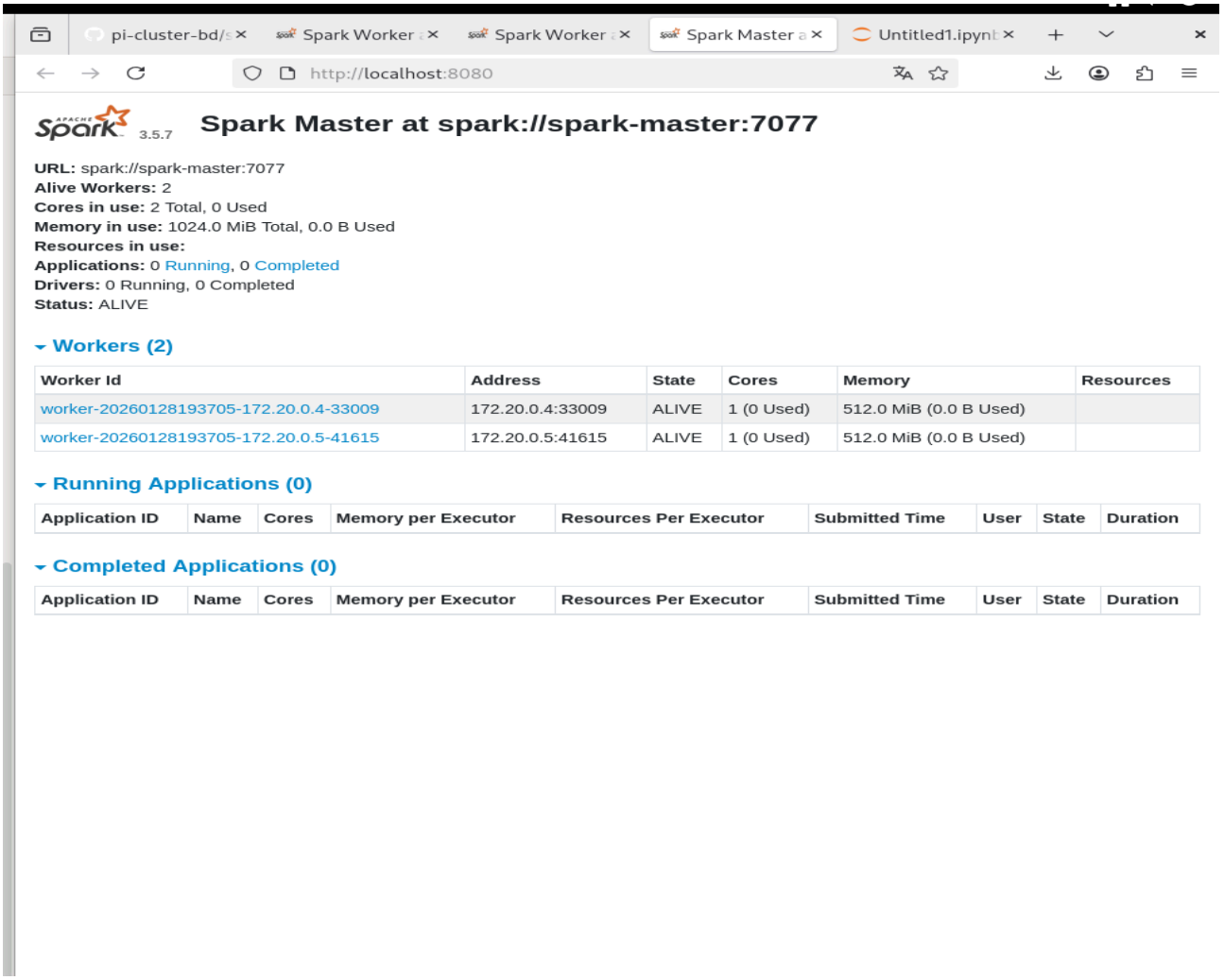
hacemos el docker compose up y monto los contenedores de jupyterlab, spark-master y dos workers

```
[+] up 21/226944e              Pull complete                                0.9s
 ✓ Image andreper/jupyterlab   Pulled                                     180.6s
 ✓ Image andreper/spark-worker Pulled                                     127.1s
 ✓ Image andreper/spark-master Pulled                                     127.1s
 ✓ Volume cluster_tfg          Created                                      0.0s
 ✓ Container spark-master      Created                                      1.3s
 ✓ Container spark-worker-1    Created                                      0.2s
 ✓ Container jupyterlab        Created                                      0.2s
 ✓ Container spark-worker-2    Created                                      0.2s
```

aqui vemos el docker de spark creado con el master, los 2 worker y el jupyterlab

El spark-master con los workers

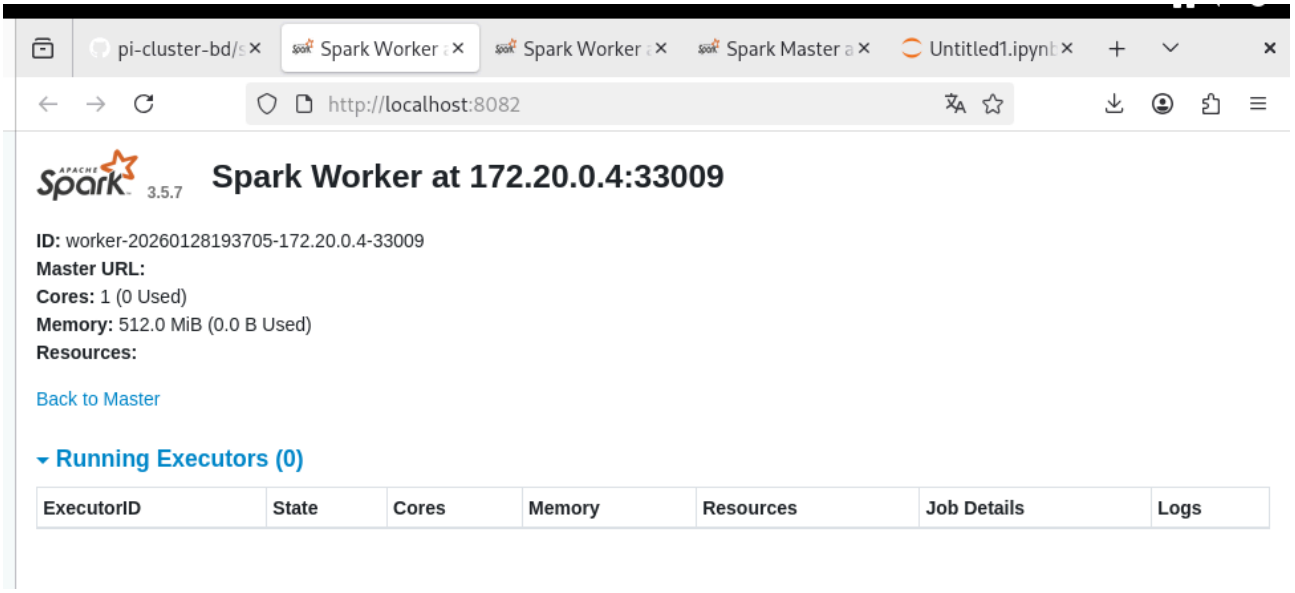
aqui vemos el master, en el master vemos los dos workers que hemos creado y que están vivos esi significa que funcionan y se detectan entre ellos



Aqui se ve en los log del master como aparece como que el a sido elegido como maestro y que se han registrado los dos workers

```
26/02/13 12:16:08 INFO Master: I have been elected leader! New state: ALIVE
26/02/13 12:16:08 INFO Master: Registering worker 172.22.0.3:46189 with 2 cores, 1024.0 MiB RAM
26/02/13 12:16:08 INFO Master: Registering worker 172.22.0.4:38691 with 2 cores, 1024.0 MiB RAM
```

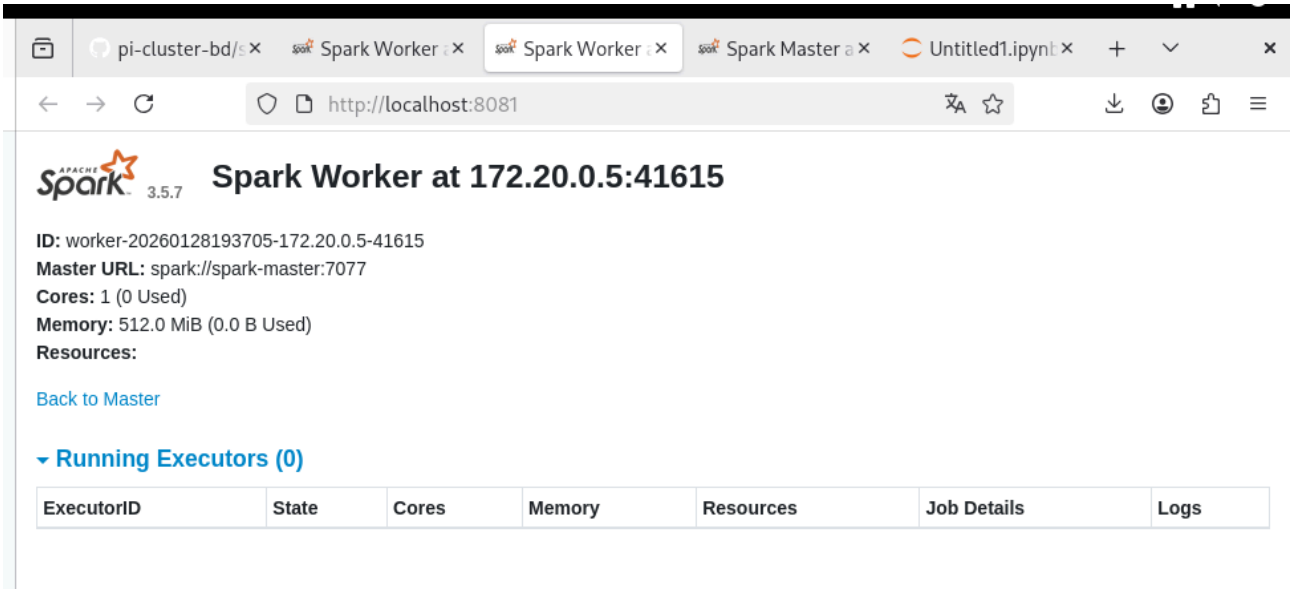
spark-worker1

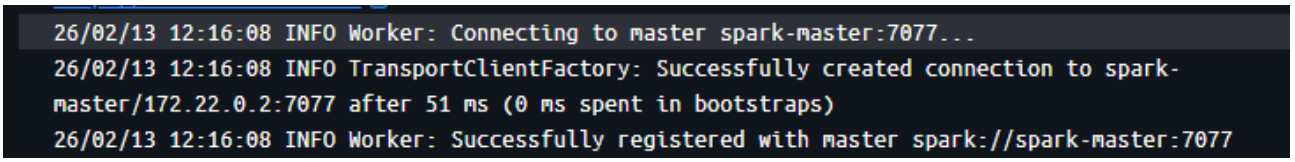


En los logs de worker sale que se intenta conectar al máster por la dirección que le hemos dado al hacerlo con el .yml en la siguiente linea ya dice que se a podido conectar y en la ultima que sea registrado

```
26/02/13 12:16:08 INFO Worker: Connecting to master spark-master:7077...
26/02/13 12:16:08 INFO TransportClientFactory: Successfully created connection to spark-
master/172.22.0.2:7077 after 46 ms (0 ms spent in bootstraps)
26/02/13 12:16:08 INFO Worker: Successfully registered with master spark://spark-master:7077
```

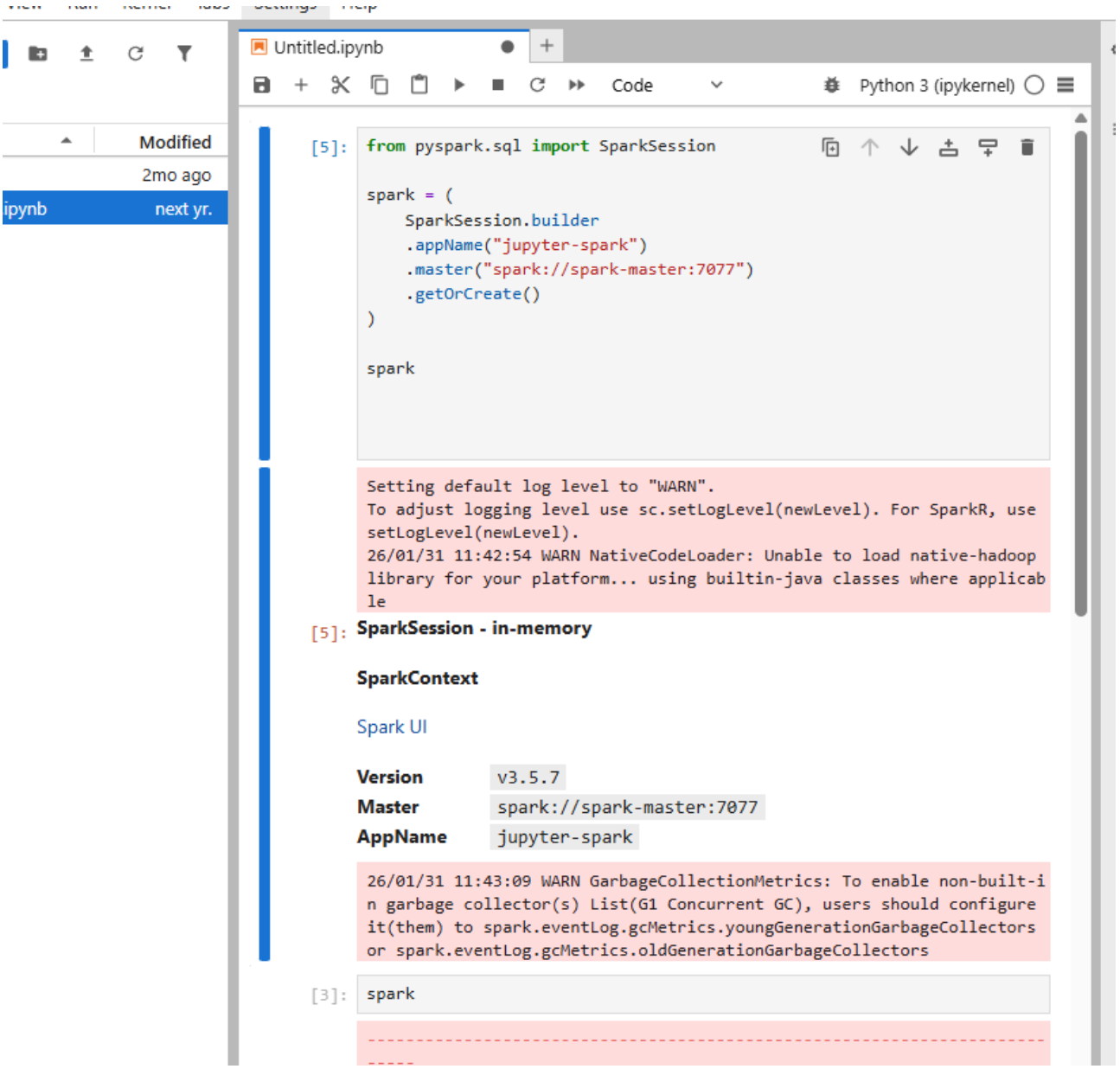
spark-worker2





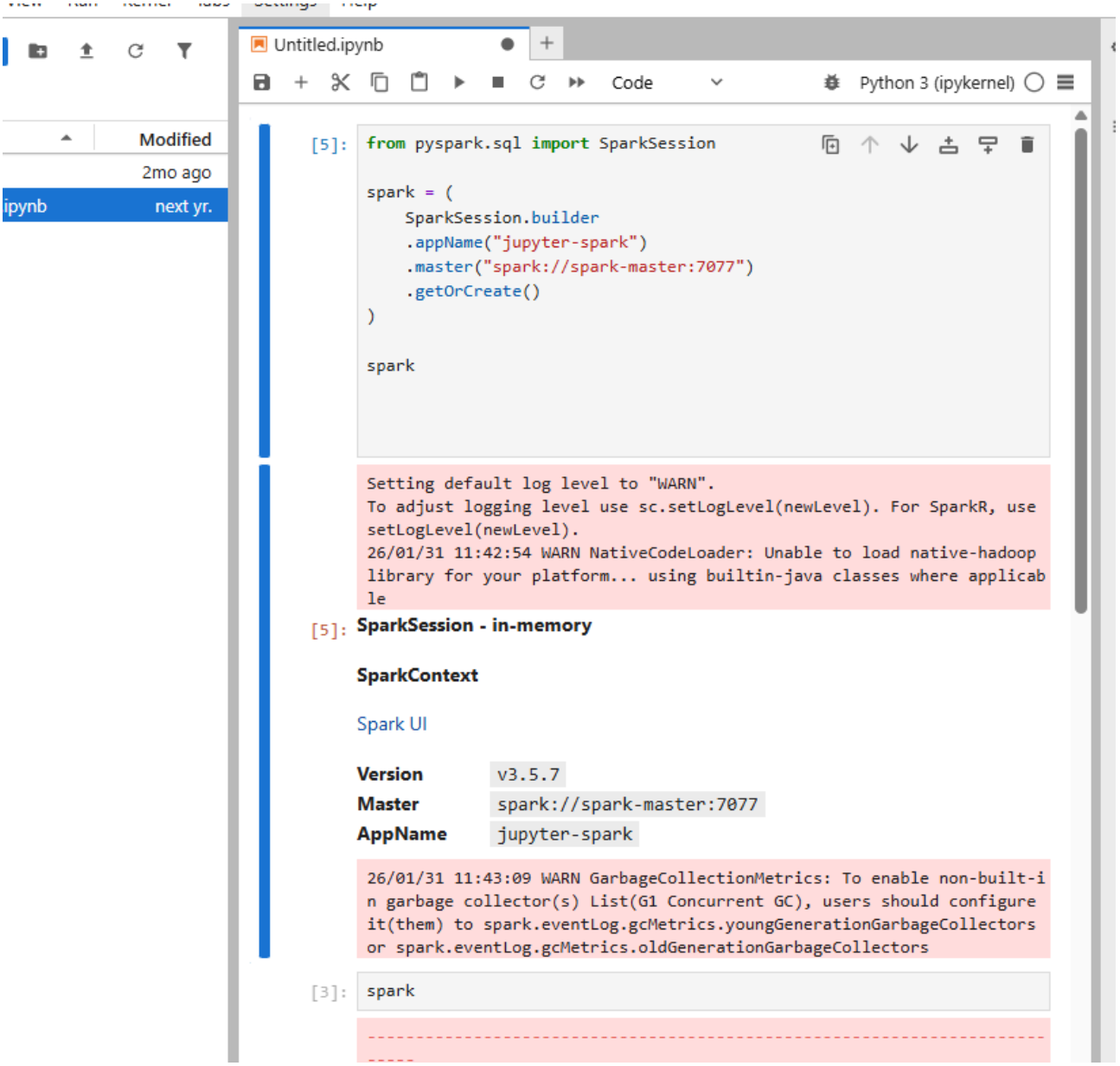
jupyterlab

en jupyterlab es donde mandamos trabajos al master y el se lo manda a los workers

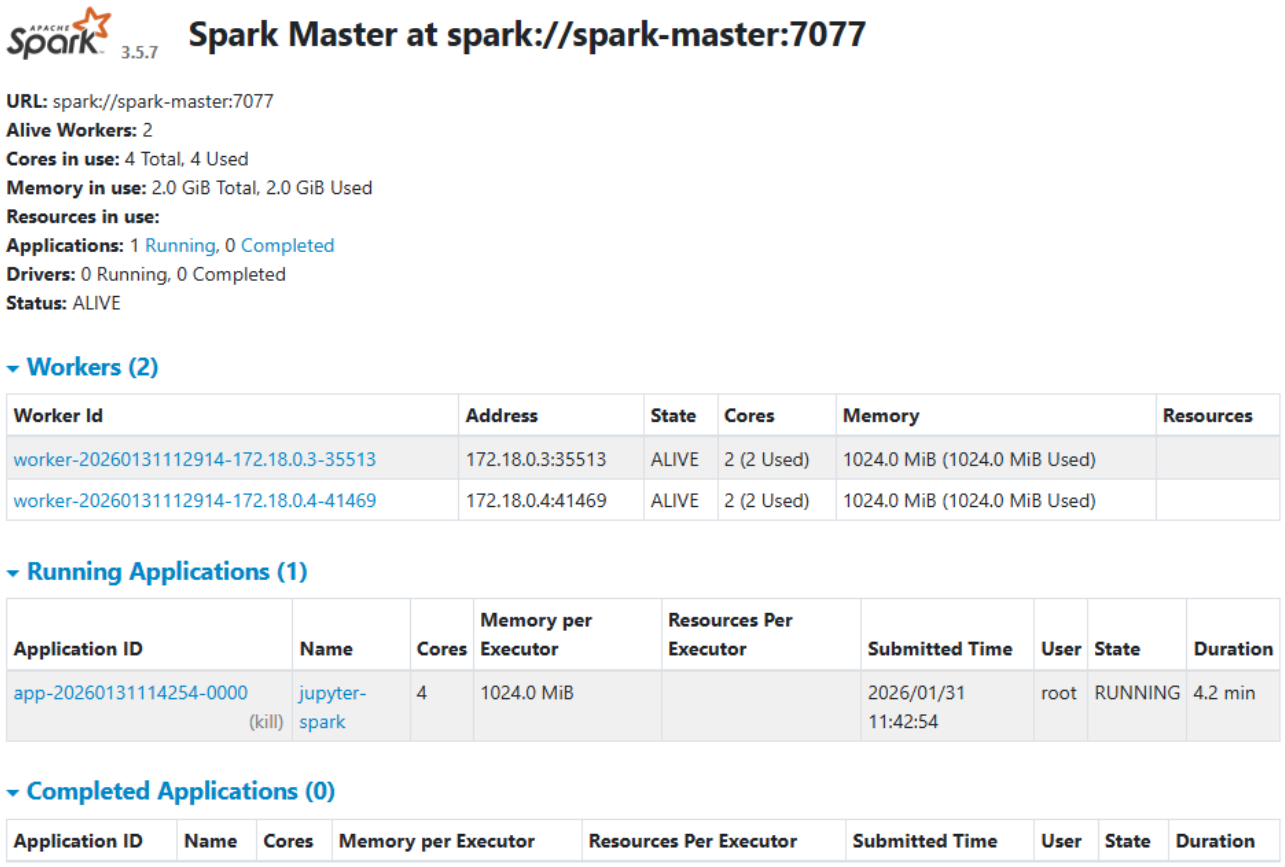

Modified
2mo ago
ipynb
next yr.
Untitled.ipynb
Code
Python 3 (ipykernel)
[5]:
from pyspark.sql import SparkSession
spark = (
SparkSession.builder
.appName("jupyter-spark")
.master("spark://spark-master:7077")
.getOrCreate()
)
spark
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/01/31 11:42:54 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
[5]:
SparkSession - in-memory
SparkContext
Spark UI
Version v3.5.7
Master spark://spark-master:7077
AppName jupyter-spark
26/01/31 11:43:09 WARN GarbageCollectionMetrics: To enable non-built-in garbage collector(s) List(G1 Concurrent GC), users should configure it(them) to spark.eventLog.gcMetrics.youngGenerationGarbageCollectors or spark.eventLog.gcMetrics.oldGenerationGarbageCollectors
[3]:
spark

creación de una sesión en pyspark

1º importando desde pyspark.sql la spark session generamos la sesión marcando el nombre de la app que vamos hacer y dónde está el master y ponemos spark al final para que muestre la sesion|.



aquí vemos que al iniciar la sesión hemos creado la primera aplicación



Spark Master at spark://spark-master:7077

3.5.7

URL: spark://spark-master:7077
Alive Workers: 2
Cores in use: 4 Total, 4 Used
Memory in use: 2.0 GiB Total, 2.0 GiB Used
Resources in use:
Applications: 1 Running, 0 Completed
Drivers: 0 Running, 0 Completed
Status: ALIVE

Workers (2)

Worker Id	Address	State	Cores	Memory	Resources
worker-20260131112914-172.18.0.3-35513	172.18.0.3:35513	ALIVE	2 (2 Used)	1024.0 MiB (1024.0 MiB Used)	
worker-20260131112914-172.18.0.4-41469	172.18.0.4:41469	ALIVE	2 (2 Used)	1024.0 MiB (1024.0 MiB Used)	

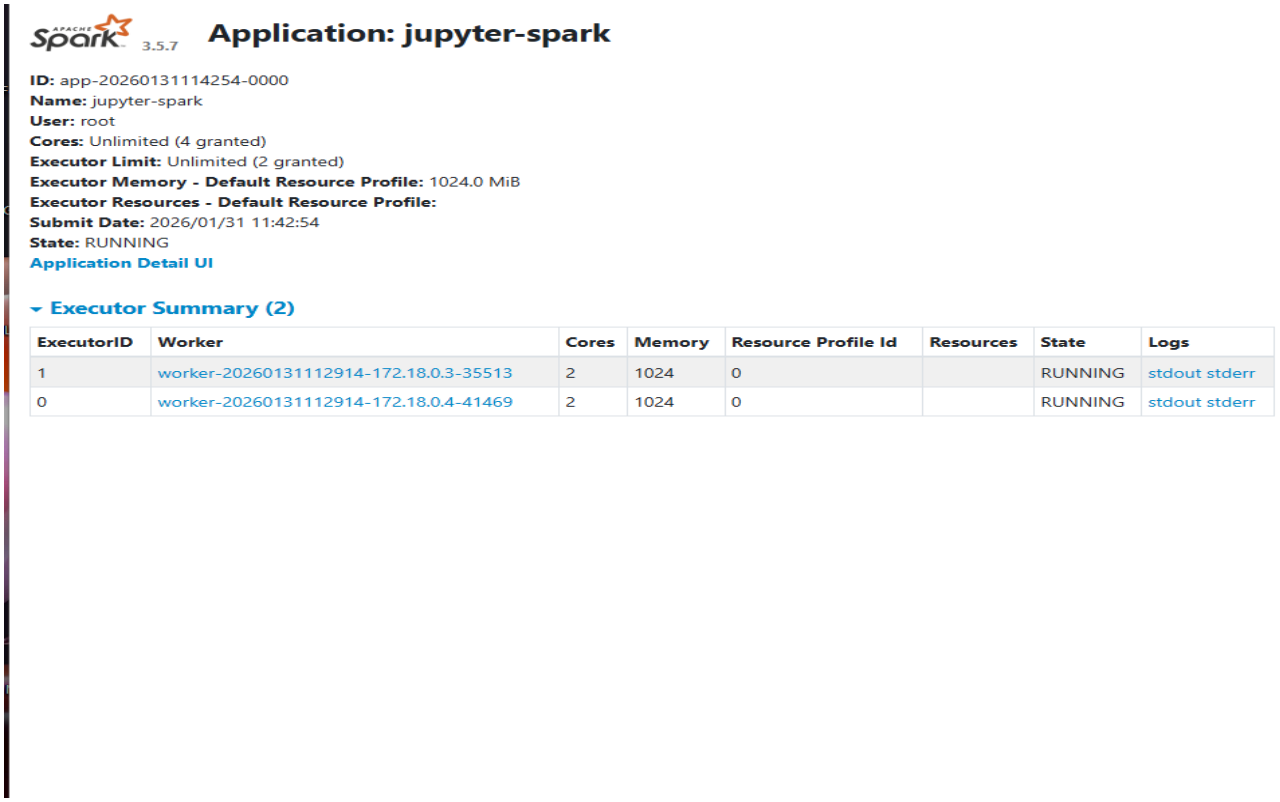
Running Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260131114254-0000 (kill)	jupyter-spark	4	1024.0 MiB		2026/01/31 11:42:54	root	RUNNING	4.2 min

Completed Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration

Aqui vemos a la aplicación y datos de ella.



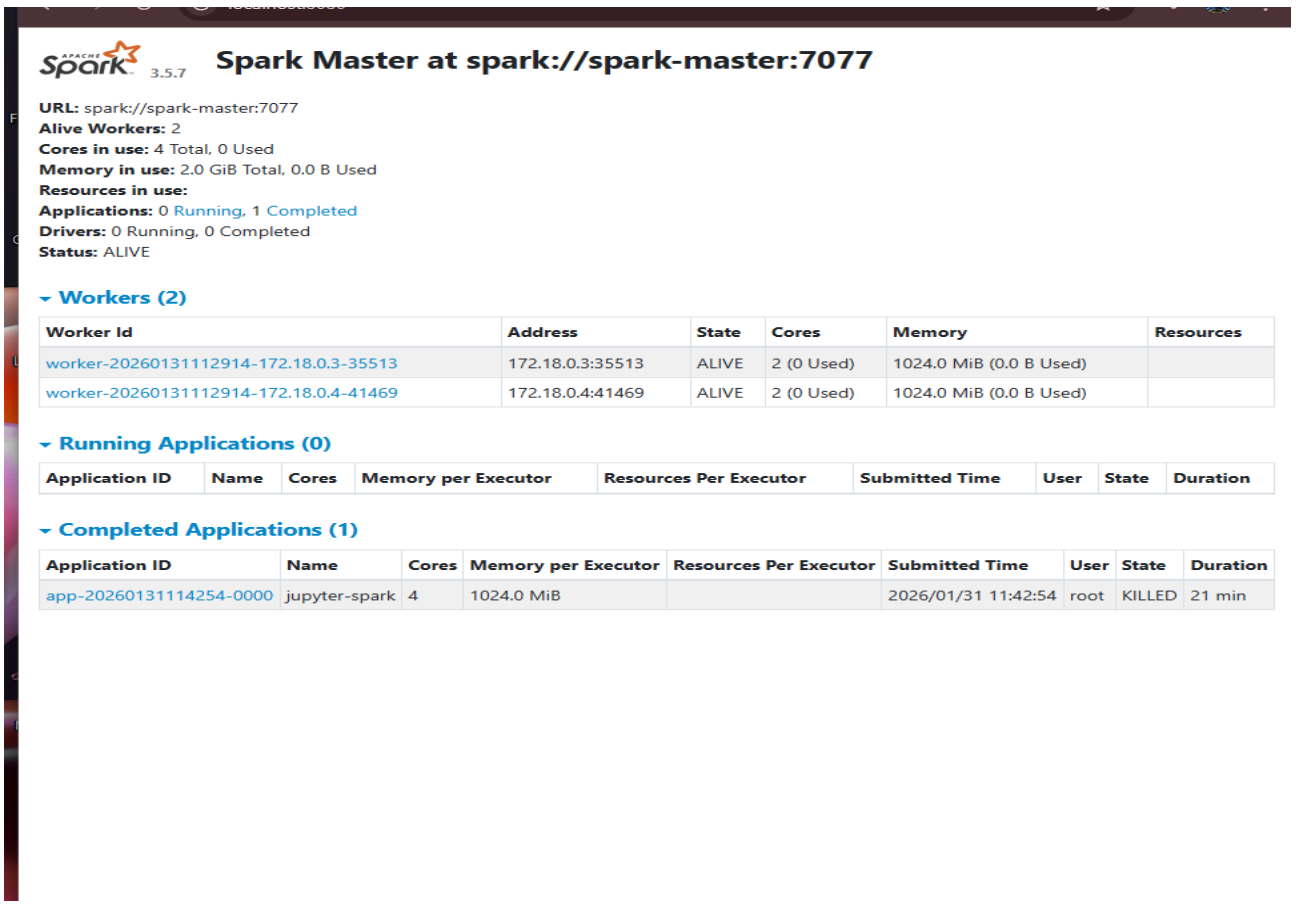
Spark 3.5.7 **Application: jupyter-spark**

ID: app-20260131114254-0000
Name: jupyter-spark
User: root
Cores: Unlimited (4 granted)
Executor Limit: Unlimited (2 granted)
Executor Memory - Default Resource Profile: 1024.0 MiB
Executor Resources - Default Resource Profile:
Submit Date: 2026/01/31 11:42:54
State: RUNNING
Application Detail UI

▾ Executor Summary (2)

ExecutorID	Worker	Cores	Memory	Resource Profile Id	Resources	State	Logs
1	worker-20260131112914-172.18.0.3-35513	2	1024	0		RUNNING	stdout stderr
0	worker-20260131112914-172.18.0.4-41469	2	1024	0		RUNNING	stdout stderr

Aquí vemos que está completa eso es cuando se mata una sesión



Spark 3.5.7 **Spark Master at spark://spark-master:7077**

URL: spark://spark-master:7077
Alive Workers: 2
Cores in use: 4 Total, 0 Used
Memory in use: 2.0 GiB Total, 0.0 B Used
Resources in use:
Applications: 0 Running, 1 Completed
Drivers: 0 Running, 0 Completed
Status: ALIVE

▾ Workers (2)

Worker Id	Address	State	Cores	Memory	Resources
worker-20260131112914-172.18.0.3-35513	172.18.0.3:35513	ALIVE	2 (0 Used)	1024.0 MiB (0.0 B Used)	
worker-20260131112914-172.18.0.4-41469	172.18.0.4:41469	ALIVE	2 (0 Used)	1024.0 MiB (0.0 B Used)	

▾ Running Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration

▾ Completed Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260131114254-0000	jupyter-spark	4	1024.0 MiB		2026/01/31 11:42:54	root	KILLED	21 min

Spark Worker at 172.18.0.4:41469

3.5.7

ID: worker-20260131112914-172.18.0.4-41469
Master URL: spark://spark-master:7077
Cores: 2 (2 Used)
Memory: 1024.0 MiB (1024.0 MiB Used)
Resources:

Back to Master

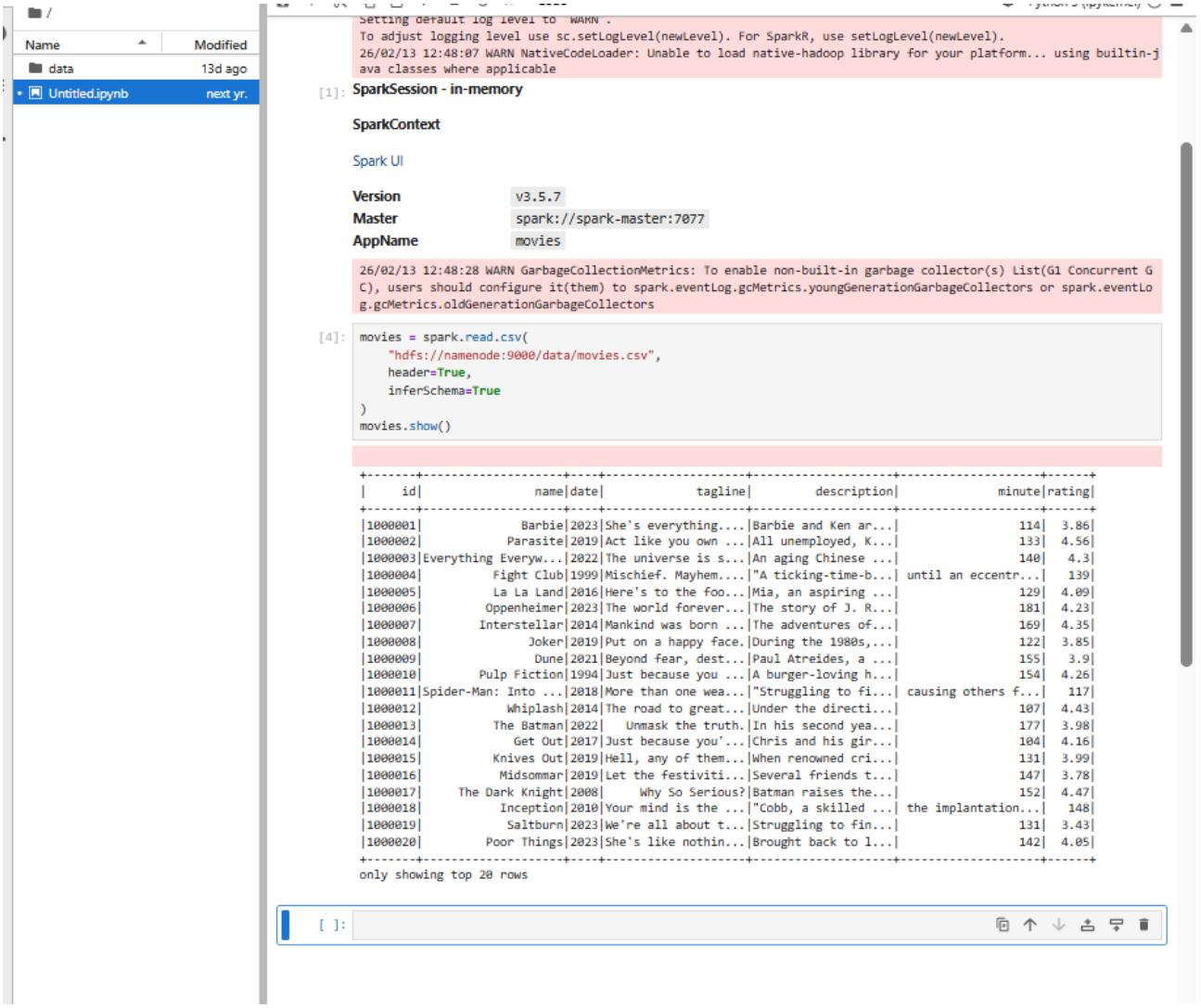
Running Executors (1)

ExecutorID	State	Cores	Memory	Resources	Job Details	Logs
0	RUNNING	2	1024.0 MiB		**ID:** app-20260131120832-0001 **Name:** movies **User:** root	stdout stderr

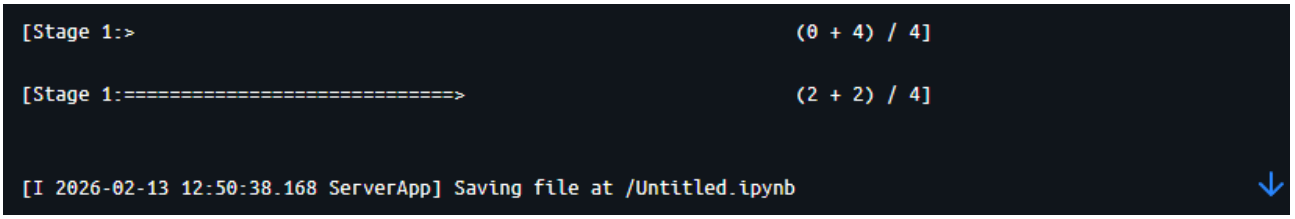
Finished Executors (1)

ExecutorID	State	Cores	Memory	Resources	Job Details	Logs
0	KILLED	2	1024.0 MiB		**ID:** app-20260131114254-0000 **Name:** jupyter-spark **User:** root	stdout stderr

Lo segundo que hacemos es decirle a spark que lea el documento dentro de hdfs el cual tenemos que poner la ruta y que meta la cabecera que es la primera fila y que las ponga del tipo que son.



Estos son los logs que han salido al ejecutarlo que es el trabajo haciéndose y que ha guardado el archivo.



CONSULTAS

pedimos que nos enseñe el nombre y la fecha de las películas si no le marcamos un número exacto en .show() siempre enseñara las primeras 20.

```
movies.select(['name', 'date']).show()
```

```
+--------------------+----+
|                name|date|
+--------------------+----+
|              Barbie|2023|
|           Parasite|2019|
|Everything Everyw...|2022|
|          Fight Club|1999|
|          La La Land|2016|
|         Oppenheimer|2023|
|        Interstellar|2014|
|               Joker|2019|
|                Dune|2021|
|        Pulp Fiction|1994|
|Spider-Man: Into ...|2018|
|            Whiplash|2014|
|          The Batman|2022|
|             Get Out|2017|
|          Knives Out|2019|
|           Midsommar|2019|
|     The Dark Knight|2008|
|           Inception|2010|
|            Saltburn|2023|
|         Poor Things|2023|
+--------------------+----+
only showing top 20 rows
```

Movies.create Or Replace Temp View: crea una vista temporal del dataframe en esta ocasión le da un nombre en este caso el nombre movies para hacer las consultas con sql.
Aquí nos seleccione el nombre de peliculas que sean del 1999.

```
movies.createOrReplaceTempView("movies")
```

```
spark.sql("select name from movies").filter(movies.date == "1999").show()
```

```
+--------------------+
|                name|
+--------------------+
|          Fight Club|
|10 Things I Hate ...|
|          The Matrix|
|Star Wars: Episod...|
|     American Beauty|
|      Eyes Wide Shut|
|   Girl, Interrupted|
| The Virgin Suicides|
|         Toy Story 2|
|     The Sixth Sense|
|      The Green Mile|
|        Notting Hill|
|            Magnolia|
|The Blair Witch P...|
|But I'm a Cheerle...|
|Being John Malkovich|
|      The Iron Giant|
|The Talented Mr. ...|
|           The Mummy|
|              Tarzan|
+--------------------+
only showing top 20 rows
```

Creando consultas en la primera pedimos que nos de las peliculas del 2006 y que empiezen por ever.

```
# Filtramos las películas donde el año de la columna 'date' sea 2006
movies_2006 = movies.filter(year("date") == 2006)

# Mostramos el resultado
movies_2006.show()
```

```
+-------+--------------------+----+--------------------+--------------------+--------------------+--------------------+
|     id|                name|date|             tagline|         description|              minute|              rating|
+-------+--------------------+----+--------------------+--------------------+--------------------+--------------------+
|1000095|Little Miss Sunshine|2006|A family on the v...|A family loaded w...|                 102|                4.17|
|1000125|The Devil Wears P...|2006|Meet Andy Sachs. ...|Andy moves to New...|                 109|                3.78|
|1000173|        The Prestige|2006|Are You Watching ...|A mysterious stor...|                 130|                4.23|
|1000199|        The Departed|2006|Lies. Betrayal. S...|To take down Sout...|                 151|                4.31|
|1000316|                Cars|2006|Ahhh... it's got ...|Lightning McQueen...|                 117|                 3.7|
|1000325|     Pan's Labyrinth|2006|What happens when...|Living with her t...|                 118|                4.16|
|1000419|     Children of Men|2006|The future's a th...|In 2027, in a cha...|                 109|                 4.3|
|1000447|    Marie Antoinette|2006|Rumor. Scandal. F...|The retelling of ...|                 123|                3.73|
|1000472|Pirates of the Ca...|2006|       Jack is back!|Captain Jack Spar...|                 151|                3.62|
|1000490|       Casino Royale|2006|Everyone has a pa...|Le Chiffre, a ban...|                 144|                4.01|
|1000529|             Paprika|2006|This is your brai...|When a machine th...|                  90|                4.08|
|1000568|Borat: Cultural L...|2006|Come to Kazakhsta...|Kazakh journalist...|                  84|                3.73|
|1000579|The Pursuit of Ha...|2006|                NULL|A struggling sale...|                 117|                3.88|
|1000585| Night at the Museum|2006|Where History Com...|Chaos reigns at t...|                 108|                3.26|
|1000641|         The Holiday|2006|It's Christmas Ev...|Two women, one fr...|                 136|                3.48|
|1000765|       She's the Man|2006|If you wanna chas...|Viola Johnson is ...|                 105|                3.34|
|1000787| High School Musical|2006|This School Rocks...|Troy, the popular...|                  98|                3.16|
|1000861|                 300|2006|Spartans, prepare...|"Based on Frank M...| where the King o...| and helped usher...|
|1000885|       Monster House|2006|The house is . . ...|Monsters under th...|                  91|                3.36|
|1000886|               Click|2006|What If You Had A...|A harried workaho...|                 107|                2.83|
+-------+--------------------+----+--------------------+--------------------+--------------------+--------------------+
only showing top 20 rows
```

```
movies.filter(col("name").startswith("ever")).show()
```

```
+-------+--------------------+----+--------------------+--------------------+--------------------+--------------------+
|     id|                name|date|             tagline|         description|              minute|              rating|
+-------+--------------------+----+--------------------+--------------------+--------------------+--------------------+
|1271003|       everyday star|2019|                NULL|Everyday states o...|                   9|                NULL|
|1273467|every dog has its...|2019|                NULL|Cult-produced med...|                   6|                NULL|
|1355669|every light retur...|2020|                NULL|Filmed on both si...|                   7|                NULL|
|1362157|      everydayPeople|2011|                NULL|Two small western...|                  66|                NULL|
|1386652|everyone gets to ...|NULL|                NULL|everyone gets to ...|                  19|                NULL|
|1403236|everything takes ...|2020|                NULL|"We see, and so w...| even dreams"" ex...| moving between t...|
|1406455|  everything must go|NULL|                NULL|Fresh off their n...|                  12|                NULL|
|1413560|everything I didn...|2023|                NULL|Atlas sees the sa...|                   5|                NULL|
|1435980|everything will b...|NULL|                NULL|Contemplating abo...|                   1|                NULL|
|1442431|everything in our...|NULL|                NULL|Part film, part e...|                   5|                NULL|
|1531003|ever present goin...|2007|                NULL|Filmmaker Philip ...|                   7|                NULL|
|1577457|everything is bea...|2022|                NULL|Queer, post-roman...|                   5|                NULL|
|1608885|  everything is fine|2021|It just doesn't a...|Following a famil...|                   8|                NULL|
|1611918|everything is ok ...|2021|                NULL|A destructive col...|                   6|                NULL|
|1669946|everyone deceives...|1992|                NULL|"Buthaina is a bu...| a thief broke  i...|                  95|
|1671870|everything finds ...|2021|                NULL|You think a bit o...|                NULL|                NULL|
|1688261|everything circle...|2023|                NULL|This poem is buil...|                   4|                NULL|
|1725743|every dove is a p...|2024|A film about the ...|every dove is a p...|                  14|                NULL|
```

Ahora vamos a hacer que lea todos los ficheros que tenemos y que nos lo muestre para que veamos que los ha leído bien.

[9]:
```
from pyspark.sql.functions import col

# Leer CSVs desde HDFS
movies = spark.read.csv("hdfs://namenode:9000/data/movies.csv",header=True, inferSchema=True)

countries = spark.read.csv("hdfs://namenode:9000/data/countries.csv",header=True, inferSchema=True)

crew = spark.read.csv("hdfs://namenode:9000/data/crew.csv",header=True, inferSchema=True)

actors = spark.read.csv("hdfs://namenode:9000/data/actors.csv",header=True, inferSchema=True)

genres = spark.read.csv("hdfs://namenode:9000/data/genres.csv",header=True, inferSchema=True)

languages = spark.read.csv("hdfs://namenode:9000/data/languages.csv",header=True, inferSchema=True)

releases = spark.read.csv("hdfs://namenode:9000/data/releases.csv",header=True, inferSchema=True)

studios = spark.read.csv("hdfs://namenode:9000/data/studios.csv",header=True, inferSchema=True)

themes = spark.read.csv("hdfs://namenode:9000/data/themes.csv",header=True, inferSchema=True)
```

[10]:
```
movies.show(5)
genres.show(5)
languages.show(5)
countries.show(5)
crew.show(5)
actors.show(5)
releases.show(5)
studios.show(5)
themes.show(5)

movies.printSchema()
genres.printSchema()
languages.printSchema()
countries.printSchema()
crew.printSchema()
actors.printSchema()
releases.printSchema()
studios.printSchema()
themes.printSchema()
```

```
+-------+--------------------+----+--------------------+--------------------+--------------------+------+
```

Aquí ponemos un nombre más pequeño para cada archivo.csv.

n [18]:
```
dfs = {
    "mov": movies, "gen": genres, "lan": languages,
    "act": actors, "cou": countries, "cre": crew,
    "rel": releases, "std": studios, "thm": themes
}
```

Es usado como un diccionario aquí hace dos cosas 1 es asegurar que los ids sean numéricos y 2 etiqueta cada dataframe para que sea identificable

9]:
```
processed_dfs = {
    name: df.withColumn("id", col("id").cast("int")).alias(name)
    for name, df in dfs.items()
}
```

Aquí combinamos todas las tablas a través de la id de cada película.

20]:
```
combined_df = processed_dfs["mov"].join(processed_dfs["lan"], "id", "inner") \
    .join(processed_dfs["gen"], "id", "inner") \
    .join(processed_dfs["act"], "id", "inner") \
    .join(processed_dfs["cou"], "id", "inner") \
    .join(processed_dfs["cre"], "id", "inner") \
    .join(processed_dfs["rel"], "id", "inner") \
    .join(processed_dfs["std"], "id", "inner") \
    .join(processed_dfs["thm"], "id", "inner")
```

Aquí asignamos un alias para evitar duplicado de nombre de las tablas.

1]:
```
from pyspark.sql.functions import col

# 1. Selección Maestra: Proyectamos todas las columnas necesarias
# Importante: Usamos los alias definidos en los joins (mov, gen, lan, etc.)
final_df = combined_df.select(
    col("mov.id").alias("id"),
    col("mov.name").alias("movie_title"),
    col("mov.date").alias("release_date"),
    col("mov.description").alias("description"),
    col("mov.minute").cast("int").alias("minute"),
    col("mov.rating").cast("float").alias("rating"),
    col("gen.genre").alias("genre"),
    col("lan.language").alias("language"),
    col("cou.country").alias("country"),
    col("std.studio").alias("studio"),
    col("thm.theme").alias("theme"),
    col("act.name").alias("actor_name")
).orderBy("id")

# 2. Registro de la vista para SQL
# Esto debe ir en una línea aparte porque devuelve None
final_df.createOrReplaceTempView("tabla_cine")

# 3. Verificación
print("Columnas registradas en tabla_cine:", final_df.columns)
final_df.show(5)
```

Esta es una consulta buscada por el nombre de una película.

```
22]: spark.sql("SELECT * FROM tabla_cine WHERE movie_title LIKE '%Barbie%'").show()
```

```
[Stage 73:=============================================>          (4 + 1) / 5]
+-------+-----------+------------+--------------------+------+------+------+--------+-------+--------------------+--------------------+------------+
|     id|movie_title|release_date|         description|minute|rating| genre|language|country|              studio|               theme|   actor_name|
+-------+-----------+------------+--------------------+------+------+------+--------+-------+--------------------+--------------------+------------+
|1000001|     Barbie|        2023|Barbie and Ken ar...|   114|  3.86|Comedy| English|     UK|LuckyChap Enterta...|Laugh-out-loud re...|Margot Robbie|
|1000001|     Barbie|        2023|Barbie and Ken ar...|   114|  3.86|Comedy| English|     UK|LuckyChap Enterta...|Amusing jokes and...|Margot Robbie|
|1000001|     Barbie|        2023|Barbie and Ken ar...|   114|  3.86|Comedy| English|     UK|LuckyChap Enterta...|Quirky and endear...|Margot Robbie|
|1000001|     Barbie|        2023|Barbie and Ken ar...|   114|  3.86|Comedy| English|     UK|LuckyChap Enterta...|Surreal and thoug...|Margot Robbie|
|1000001|     Barbie|        2023|Barbie and Ken ar...|   114|  3.86|Comedy| English|     UK|LuckyChap Enterta...|Emotional and cap...|Margot Robbie|
|1000001|     Barbie|        2023|Barbie and Ken ar...|   114|  3.86|Comedy| English|     UK|LuckyChap Enterta...|Moving relationsh...|Margot Robbie|
|1000001|     Barbie|        2023|Barbie and Ken ar...|   114|  3.86|Comedy| English|     UK|LuckyChap Enterta...|Crude humor and s...|Margot Robbie|
|1000001|     Barbie|        2023|Barbie and Ken ar...|   114|  3.86|Comedy| English|     UK|LuckyChap Enterta...|Humanity and the ...|Margot Robbie|
|1000001|     Barbie|        2023|Barbie and Ken ar...|   114|  3.86|Comedy| English|     UK|        Heyday Films|Laugh-out-loud re...|Margot Robbie|
|1000001|     Barbie|        2023|Barbie and Ken ar...|   114|  3.86|Comedy| English|     UK|        Heyday Films|Amusing jokes and...|Margot Robbie|
|1000001|     Barbie|        2023|Barbie and Ken ar...|   114|  3.86|Comedy| English|     UK|        Heyday Films|Quirky and endear...|Margot Robbie|
|1000001|     Barbie|        2023|Barbie and Ken ar...|   114|  3.86|Comedy| English|     UK|        Heyday Films|Surreal and thoug...|Margot Robbie|
|1000001|     Barbie|        2023|Barbie and Ken ar...|   114|  3.86|Comedy| English|     UK|        Heyday Films|Emotional and cap...|Margot Robbie|
|1000001|     Barbie|        2023|Barbie and Ken ar...|   114|  3.86|Comedy| English|     UK|        Heyday Films|Moving relationsh...|Margot Robbie|
|1000001|     Barbie|        2023|Barbie and Ken ar...|   114|  3.86|Comedy| English|     UK|        Heyday Films|Crude humor and s...|Margot Robbie|
|1000001|     Barbie|        2023|Barbie and Ken ar...|   114|  3.86|Comedy| English|     UK|        Heyday Films|Humanity and the ...|Margot Robbie|
|1000001|     Barbie|        2023|Barbie and Ken ar...|   114|  3.86|Comedy| English|     UK|      NB/GG Pictures|Laugh-out-loud re...|Margot Robbie|
|1000001|     Barbie|        2023|Barbie and Ken ar...|   114|  3.86|Comedy| English|     UK|      NB/GG Pictures|Amusing jokes and...|Margot Robbie|
|1000001|     Barbie|        2023|Barbie and Ken ar...|   114|  3.86|Comedy| English|     UK|      NB/GG Pictures|Quirky and endear...|Margot Robbie|
|1000001|     Barbie|        2023|Barbie and Ken ar...|   114|  3.86|Comedy| English|     UK|      NB/GG Pictures|Surreal and thoug...|Margot Robbie|
+-------+-----------+------------+--------------------+------+------+------+--------+-------+--------------------+--------------------+------------+
only showing top 20 rows
```

5-Resultados y Análisis

El resultado al que hemos llegado es tener un cluster de spark para el proceso de datos. Por jupyterlab hacemos la sesión en spark, las consultas que hacemos para buscar lo que queremos sacar de una base de datos que analizamos y en hdfs hemos almacenado los datos que hemos utilizado en nuestro cluster.

6-Conclusiones Recomendaciones

En conclusión hemos hecho un proyecto a una escala pequeña al hacerlo en docker ya que esto se haría en una empresa con varios nodos, con una cantidad de workers suficientes para no obstruir lo red y con base de datos más extensa y consultas más complejas que las que hemos hecho en este proyecto.

Mejoras: en vez de hacerlo en un docker hacer que cada nodo sea un ordenador/servidor con sus propios recursos así haciendo que todo no tire de los recursos de una misma máquina así

potenciando el nivel del procesado de datos y de almacenamiento y más seguridad a la hora de perdida de datos porque falle un servidor llaque los datos se replican en varios bloque de datanode.

7-Bibliografía

https://github.com/pacotoh/bd-ia-courses/blob/main/notebooks/bda/IntroPySpark.ipynb
https://hub.docker.com/u/andreper
https://hub.docker.com/r/gchq/hdfs
https://www.ibm.com/es-es/think/topics/hdfs
https://www.ibm.com/es-es/think/topics/apache-spark

8-Anexos